

API Gateway

Go · Redis · Docker

Diagrams & Workflow Reference

Diagram 1
System Overview

Diagram 2
Request Lifecycle

Diagram 3
Token Bucket

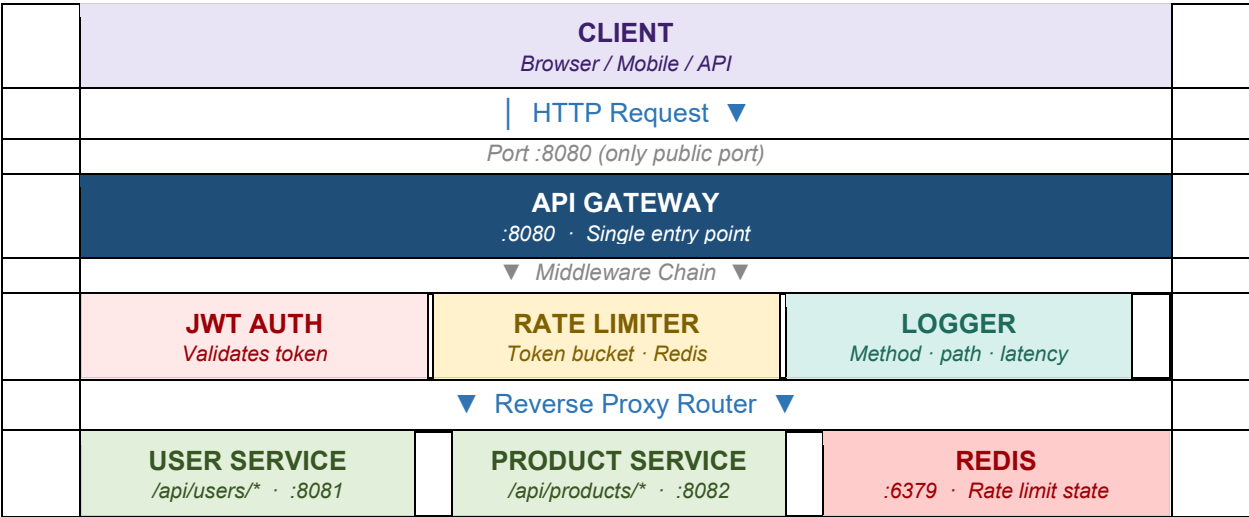
Diagram 4
Docker Layout

Diagram 5
JWT Flow

Diagram 6
Middleware
Order

Diagram 1 — System Overview

This diagram shows every component in the project and how they connect. The client only ever talks to the gateway on port 8080. Everything behind that — the microservices and Redis — is hidden inside Docker's internal network.





Key insight — The client has no idea whether one microservice exists behind the gateway or twenty. That abstraction is the entire point of the gateway pattern.

Component summary

Component	Role
API Gateway :8080	Single entry point — auth, rate limit, log, route
JWT Auth Middleware	Validates every incoming token; rejects with 401 if invalid
Rate Limiter Middleware	Token bucket per client IP; rejects with 429 when empty
Logger Middleware	Records method, path, status, latency for every request
Reverse Proxy Router	Matches URL prefix → forwards to correct microservice
User Service :8081	REST API handling /api/users/* endpoints
Product Service :8082	REST API handling /api/products/* endpoints
Redis :6379	Shared state store for token bucket counts

Diagram 2 — Request Lifecycle

Every single HTTP request the gateway receives follows this exact path. There are two early exit points where the request is rejected before it ever reaches a microservice.



Two failure paths — A request with no token returns 401 immediately — rate limiting is never even checked. A request with a valid token but an exhausted bucket returns 429. Only requests that clear both gates are forwarded.

What happens at each decision point

Gate	What is checked
JWT check	Does the Authorization header exist? Is the Bearer token a valid JWT signed with the server's secret key? Is it expired?

Gate	What is checked
Rate limit check	Does Redis have a key for this client IP? Is the token count above zero? If yes, decrement and proceed.
URL routing	Does the path start with /api/users/? Forward to user-service. /api/products/? Forward to product-service.

Diagram 3 — Token Bucket Rate Limiting

The token bucket is the core algorithm behind the rate limiter. Each client IP has its own bucket stored as a key in Redis. The bucket refills over time and drains with each request.

Bucket states

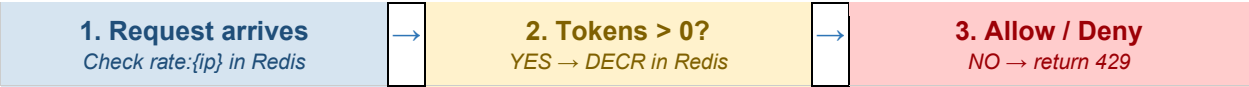


Visual: 10-token bucket (full)

Each circle represents one available request token:



The 3-step flow for every request



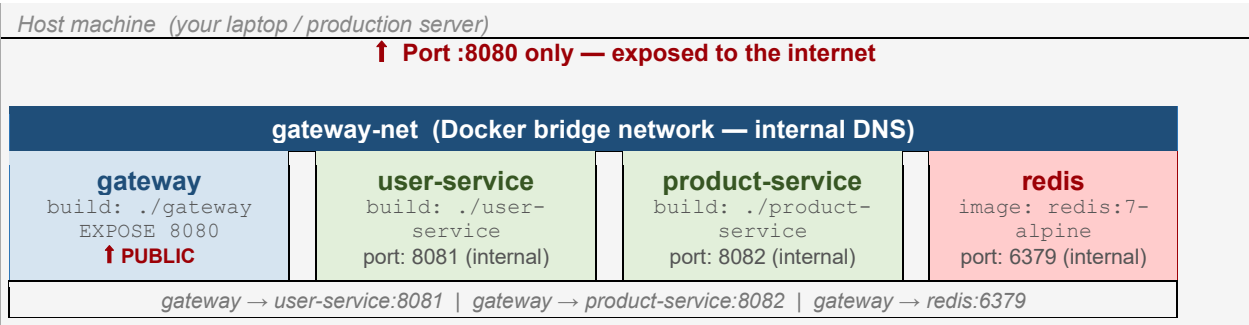
Why Redis? — If rate limiting were stored in memory inside the gateway process, running two gateway containers would give each client double the limit. Redis is a shared external store, so all gateway instances read and write the same counters — true distributed rate limiting.

Refill behaviour

Redis keys are set with a TTL (time-to-live). When the TTL expires, the key is deleted — the next request recreates it at full capacity. This is the refill mechanism. A bucket of 10 tokens with a 1-second TTL means 10 requests per second per IP.

Diagram 4 — Docker Compose Architecture

Docker Compose starts all four containers and connects them through a shared bridge network called gateway-net. Only the gateway container exposes a port to the outside world.



How containers talk to each other

Inside gateway-net, each container is reachable by its service name as a hostname. Docker handles the DNS automatically.

From container	Reaches via
gateway	redis:6379 (for rate limit state)
gateway	user-service:8081 (for /api/users/* requests)
gateway	product-service:8082 (for /api/products/* requests)
user-service	No outbound connections needed
product-service	No outbound connections needed
redis	No outbound connections needed

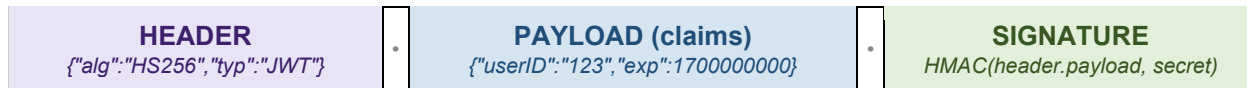


Isolation — The microservices are not reachable from the internet at all — only from inside gateway-net. This means even if an attacker knew the product-service existed, they could not reach it directly.

Diagram 5 — JWT Authentication Flow

A JWT (JSON Web Token) is a self-contained credential. It has three base64-encoded sections separated by dots. The server never needs to look up a database to validate it — the signature is the proof.

Token structure



What each part contains

Part	Contents
Header	Algorithm and token type: {"alg": "HS256", "typ": "JWT"}
Payload	User claims: {"userID": "123", "exp": 1700000000} — expiry is Unix timestamp
Signature	HMAC-SHA256(base64(header) + "." + base64(payload), secretKey)

Validation steps in the gateway



The secret key — The secret key never travels in the token. The client only ever sees the signature. The server recomputes the HMAC using its own copy of the key — if the result matches the signature in the token, the token is authentic.

Why JWT instead of sessions?

- **Sessions:** Server stores session in memory or DB. Hard to scale across multiple gateway instances.
- **JWT:** All user data is in the token itself. Any gateway instance can validate it with just the secret key.
- **Trade-off:** JWTs cannot be invalidated before expiry. Use short expiry (15–60 min) and refresh tokens for production.

Diagram 6 — Middleware Execution Order

In Gin (and most Go HTTP frameworks), middleware is executed in the exact order it is registered. This order matters — auth must come before rate limiting, and both must come before routing.

1	REQUEST IN	HTTP arrives at :8080
	▼	
2	LOGGER — start timer	Records start time for latency measurement
	▼	
3	JWT MIDDLEWARE	Parse header → verify signature → c.Next() or Abort(401)
	▼	
4	RATE LIMIT MIDDLEWARE	Redis GET tokens → DECR → c.Next() or Abort(429)
	▼	
5	REVERSE PROXY HANDLER	Match path → forward to microservice → await response
	▼	
6	LOGGER — record result	Writes: status code · response time · path
	▼	
7	RESPONSE OUT	Final response returned to client



Why this order? — Auth first ensures that unauthenticated requests are rejected before they consume a rate limit token. Rate limit second ensures that even valid users cannot spam the system. Logger wraps everything so it can capture both the start time and the final response code.

What c.Next() and c.Abort() do

Call	Effect
c.Next()	Passes control to the next middleware or handler in the chain
c.Abort()	Stops the chain — no further middleware or handler runs
c.AbortWithStatusJSON(401, ...)	Stops chain AND writes a 401 response immediately
c.AbortWithStatusJSON(429, ...)	Stops chain AND writes a 429 response immediately

End-to-End Workflow Summary

This is the complete picture — from a client typing a URL to receiving a response:

#	Stage	What happens
1	Client sends request	HTTP GET /api/users with Authorization: Bearer <token> header
2	JWT validation	Gateway extracts token, parses it, verifies HMAC. Invalid → 401.
3	Rate limit check	Redis checked for rate:{ip}. Bucket empty → 429. Otherwise decrement.
4	Request logged	Start time recorded. Method and path noted.
5	URL routing	Path prefix matched. Request forwarded to the correct microservice.
6	Microservice handles	User-service or product-service processes the request, returns JSON.
7	Response travels back	Proxy streams the response back through the gateway to the client.
8	Logger records result	Status code and elapsed time written to log.

Use the diagrams in this document as your build blueprint. Start with Diagram 1 to understand the system, then Diagram 2 to understand the request flow, and work through each one as you build.